PHISHGUARD NEXUS

TECHNICAL PROJECT REFERENCE & SPECIFICATION MANUAL

Comprehensive System Reference, Codebase Audit, API Documentation, ML Pipeline Specification, and Viva Guide

Target Audience:	Students, Faculty Members, Developers, Security Analysts, Examiners
Project Scope:	Full-Stack AI Cybersecurity Platform for Phishing URL Detection & XAI
Frontend Stack:	React v18.3.1, Vite v5.4.21, JavaScript JSX, CSS Modules
Backend Stack:	Python 3.11, FastAPI v0.115.0, Uvicorn, Pydantic v2
Machine Learning:	scikit-learn v1.5.1 (4 Models: Logistic Regression, DT, RF, GB)
Database Layer:	SQLite 3, SQLAlchemy ORM v2.0.32 (4 Tables)
Test Suite:	Pytest v9.1.1 (148 Passed Tests, 1 Skipped, 0 Failures)
Primary OS Environment:	Windows PowerShell (Compatible with Linux/macOS)

Strict Factual Compliance: All paths, routes, features, dependency versions, and commands documented in this manual were empirically verified from the active repository.

1.0 PROJECT OVERVIEW

PhishGuard Nexus is a full-stack cybersecurity application engineered for real-time phishing website detection, structural URL feature extraction, multi-model machine learning classification, pseudo-SHAP explainability, threat case management, and experimental model evaluation. The platform addresses the critical vulnerability of traditional blacklists—which fail to identify zero-day or dynamically generated phishing links—by performing 100% offline lexical analysis directly on URL character strings.

The system operates via a decoupled client-server architecture: the user submits a candidate URL through a React 18 SPA interface. The request is dispatched over HTTP REST endpoints to a FastAPI asynchronous backend. The backend validates the input, calculates 22 canonical structural features, runs ensemble machine learning inference across four verified classifiers (Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting), maps the resulting ensemble probability to a 5-tier Risk Score (0–100), generates human-interpretable feature impact explanations, and persists the complete telemetry into an SQLite relational database via SQLAlchemy ORM.

2.0 TECHNOLOGY STACK

Frontend Core	React	JavaScript / JSX	Declarative single-page application UI component rendering.
Frontend Router	React Router DOM	v6.26.0	Client-side routing across 13 dedicated workspace pages.
Frontend Build	Vite	v5.4.2	Development HMR server & production Rollup bundler.
HTTP Client	Axios	v1.7.4	Asynchronous REST API requests to FastAPI backend.
Backend Framework	FastAPI	Python 3.11	Asynchronous REST API server & router dispatching.
ASGI Server	Uvicorn	v0.30.0	High-performance ASGI server for hosting FastAPI.
Machine Learning	scikit-learn	v1.5.1 / Python	Model training, scaler normalization & inference execution.
Model Persistence	joblib	v1.4.2	Serializing & loading trained `.joblib` model binaries.
Database / ORM	SQLite / SQLAlchemy	SQLAlchemy v2.0.32	Relational ORM schema management and data persistence.
Testing Suite	Pytest / pytest-asyncio	v9.1.1	Automated unit, feature parity, API & database testing.

3.0 PROGRAMMING LANGUAGES

JavaScript (ES6+) & JSX: Used across `frontend/src` for component logic, React Hooks, Axios state management, DOM event handlers, and UI rendering.

Python 3.11: Primary backend language used for FastAPI routing (`backend/app/routers`), SQLAlchemy models (`backend/app/models`), URL validation (`url_validator.py`), 22-feature extraction (`feature_extractor.py`), ML inference (`ml_predictor.py`), and ML pipeline training scripts (`ml/*.py`).

SQL / SQLite: Relational database language embedded via SQLAlchemy ORM for schema definition, querying, and updating tables (`scans`, `cases`, `audit_logs`, `research_experiments`).

CSS / Vanilla CSS: Custom stylesheet styling (`index.css`, component styles) defining color variables, dark theme aesthetics, grid layouts, and visual animations.

4.0 FRONTEND & DEPENDENCY REFERENCE

react	^18.3.1	UI Library	Core component view rendering and lifecycle hooks.
react-dom	^18.3.1	DOM Renderer	Mounting React application tree to HTML root element.
react-router-dom	^6.26.0	Client Router	SPA navigation across 13 pages without page reloads.
recharts	^2.12.7	Charting Library	Interactive radar charts, bar charts & trend analytics.
axios	^1.7.4	HTTP Client	Dispatched REST API requests to FastAPI endpoints.
react-icons	^5.3.0	Icon Library	Cybersecurity vector icons (shields, alerts, checkmarks).
framer-motion	^11.3.24	Animation Engine	Smooth page transitions, modal slides & gauge animations.
vite	^5.4.2	Build Tooling	Development dev server & production JS/CSS bundler.

5.0 FRONTEND DIRECTORY STRUCTURE

```
frontend/ ■■■ package.json # Node.js dependencies & scripts (dev, build, preview) ■■■ vite.config.js # Vite bundler configuration & proxy settings ■■■ index.html # HTML5 document entry point ■■■ src/ ■■■ main.jsx # React DOM root render entry ■■■ App.jsx # Top-level router wrapper & layout structure ■■■ index.css # Global CSS variables, theme tokens & baseline styles ■■■ components/ # Reusable UI component modules ■ ■■■ fusion/ # Intelligence fusion & decision assistance panels ■ ■■■ layout/ # Header, Sidebar, and navigation components ■ ■■■ nexus/ # Command palette, audit viewer, system monitor ■ ■■■ threat/ # Threat profile, radar, URL DNA, timeline widgets ■■■ context/ # PreferencesContext for global state ■■■ pages/ # 13 primary application page components ■■■ services/ # API HTTP client integration (api.js, threatIntelligence.js) ■■■ utils/ # Evidence fusion algorithms (evidenceFusion.js)
```

6.0 FRONTEND PAGES REFERENCE

HomePage.jsx	/	Hero banner, quick URL scan bar, overview feature cards.	POST /api/scan
ScannerPage.jsx	/scanner	Primary URL security scanner, live SHAP breakdown, top reasons.	POST /api/scan
DashboardPage.jsx	/dashboard	SOC summary metrics, recent scans widget, threat distribution.	GET /api/history, GET /api/cases/stats
HistoryPage.jsx	/history	Searchable scan log table, risk filters, single/bulk deletion.	GET /api/history, DELETE /api/history
CasesPage.jsx	/cases	Threat case management (PG-XXXXX), status change, notes.	GET /api/cases, POST /api/cases
InvestigationPage.jsx	/investigation/:id	Deep-dive investigation workspace, evidence fusion, notes.	GET /api/cases/{id}, PUT /api/cases/{id}
ComparePage.jsx	/compare	Side-by-side comparison of two URLs or ML model metrics.	POST /api/scan, GET /api/models/compare
ResearchLabPage.jsx	/research	Interactive ML lab for cross-validation, ROC curves, leakage check.	GET /api/research/*
SimulatorPage.jsx	/simulator	Offline URL modification laboratory for threat testing.	POST /api/scan

ModelsPage.jsx	/models	Detailed view of 4 ML models, hyperparams & importances.	GET /api/models/compare
AnalyticsPage.jsx	/analytics	Aggregated threat metrics, keyword charts & TLD breakdown.	GET /api/analytics
SettingsPage.jsx	/settings	Risk threshold controls, animation toggles, theme preferences.	Local Storage / PreferencesContext
AboutPage.jsx	/about	Project background, academic credits, research citations.	None (Static content)

7.0 BACKEND STRUCTURE & API SPECIFICATION

```
backend/ ■■■ main.py # FastAPI application entry point, CORS & startup handlers ■■■ app/ ■ ■■■ config.py # Central configuration, paths, model weights & risk thresholds ■ ■■■ database.py # SQLite engine, SessionLocal & automatic database migration ■ ■■■ models/ # SQLAlchemy ORM database models ■ ■ ■■■ scan.py # Scan record ORM model ■ ■ ■■■ case.py # Threat case ORM model ■ ■ ■■■ audit_log.py # Audit log ORM model ■ ■ ■■■ research.py # Research experiment ORM model ■ ■■■ schemas/ # Pydantic request/response validation schemas ■ ■ ■■■ scan.py # Scan request & response models ■ ■ ■■■ case.py # Case management validation schemas ■ ■■■ services/ # Core business logic services ■ ■ ■■■ url_validator.py # Strict RFC 3986 URL format validator ■ ■ ■■■ feature_extractor.py # 22-feature extraction engine ■ ■ ■■■ ml_predictor.py # Multi-model prediction & weighting engine ■ ■ ■■■ explainer.py # Pseudo-SHAP explainability engine ■ ■ ■■■ research_service.py # Research lab statistical calculations ■ ■ ■■■ report_generator.py # Executive report summary formatter ■ ■■■ routers/ # FastAPI API REST route endpoints ■ ■■■ scanner.py # POST /api/scan ■ ■■■ history.py # GET/DELETE /api/history ■ ■■■ cases.py # Case management endpoints ■ ■■■ models.py # Model metrics & feature importance ■ ■■■ research.py # Research lab REST endpoints ■ ■■■ audit_logs.py # Security audit log endpoints
```

7.1 Complete REST API Endpoints Table (32 Endpoints Total)

scanner.py	POST	/api/scan	Extract 22 features, evaluate 4 ML models, return risk score & SHAP.
history.py	GET	/api/history	Retrieve list of past scans sorted by creation timestamp.
history.py	DELETE	/api/history	Purge all stored scan records from SQLite database.
history.py	DELETE	/api/history/{scan_id}	Delete single scan record by ID.
history.py	GET	/api/analytics	Get aggregated threat metrics, TLD counts & risk distributions.
history.py	GET	/api/report/{scan_id}	Generate structured text report summary for specific scan.
cases.py	POST	/api/cases	Escalate scan into formal threat case with PG-XXXXX ID.
cases.py	GET	/api/cases	Retrieve all threat cases with status & severity filters.
cases.py	GET	/api/cases/stats	Get summary statistics of active, investigating & resolved cases.
cases.py	GET	/api/cases/{case_id}	Retrieve single case details by ID.
cases.py	PUT	/api/cases/{case_id}	Update analyst notes, investigation assessment, or resolution.
cases.py	DELETE	/api/cases/{case_id}	Delete single threat case record.
cases.py	POST	/api/cases/{case_id}/reopen	Reopen resolved case back to INVESTIGATING status.
cases.py	GET	/api/cases/{case_id}/report	Export detailed case investigation report.
models.py	GET	/api/models/compare	Return accuracy, precision, recall, F1 & AUC-ROC for 4 models.
models.py	GET	/api/models/feature-importance	Return feature importance weights for Random Forest & Decision Tree.
research.py	GET	/api/research/datasets	List available dataset modes (synthetic vs real_world).
research.py	GET	/api/research/leakage-check	Execute train/test leakage audit across features.
research.py	GET	/api/research/model-comparison	Fetch cross-model evaluation summary tables.
research.py	POST	/api/research/cross-validation	Execute 5-fold cross-validation on specified model.
research.py	POST	/api/research/threshold-analysis	Calculate precision-recall trade-offs across 0.0-1.0 cutoffs.
research.py	GET	/api/research/roc-curve	Generate FPR/TPR coordinates for rendering ROC curves.
research.py	GET	/api/research/pr-curve	Generate Precision-Recall curve coordinates.
research.py	GET	/api/research/feature-importance	Fetch research-level feature ranking vectors.
research.py	POST	/api/research/robustness	Execute noise injection test to measure model stability.
research.py	POST	/api/research/stability	Execute seed perturbation testing.
research.py	GET	/api/research/experiments	List saved research experiment runs from DB.
research.py	POST	/api/research/experiments	Save new research experiment record to database.
research.py	GET	/api/research/experiments/{id}	Fetch single experiment record by EXP-XXXXX ID.
research.py	GET	/api/research/research-summary	Get high-level summary of lab benchmark runs.
audit_logs.py	GET	/api/audit-logs	Fetch security event audit trail.
audit_logs.py	POST	/api/audit-logs	Record manual security action to audit log.

8.0 PRACTICAL COMMANDS & SETUP CHEAT SHEET

WINDOWS POWERSHELL SETUP (PRIMARY ENVIRONMENT):

1. Backend Setup & Startup:
cd "d:\mini project 5th sem\phishguard\backend" python -m venv .venv .\.venv\Scripts\Activate.ps1 pip install -r requirements.txt uvicorn app.main:app --reload --port 8000

2. Frontend Setup & Startup:
cd "d:\mini project 5th sem\phishguard\frontend" npm install npm run dev

3. Automated Verification Testing:
cd "d:\mini project 5th sem\phishguard" python -m pytest tests/ cd frontend && npm run build

LINUX / MAC TERMINAL COMMANDS:

cd phishguard/backend && python3 -m venv .venv && source .venv/bin/activate pip install -r requirements.txt && uvicorn app.main:app --reload cd ../frontend && npm install && npm run dev

9.0 VIVA VOCE QUICK REFERENCE MANUAL

Q: What is PhishGuard Nexus?

A: PhishGuard Nexus is an intelligent cybersecurity application combining lexical URL feature extraction, 4-model machine learning ensemble classification, pseudo-SHAP explainability, and SOC threat case management to detect phishing links without opening target web pages.

Q: Why extract features from the URL string instead of scraping webpage DOM content?

A: URL string analysis is 100% offline, executes in milliseconds, requires minimal memory, and eliminates Server-Side Request Forgery (SSRF) and remote code execution vulnerabilities inherent in fetching untrusted external websites.

Q: Which 4 ML models are used and why?

A: Logistic Regression (linear baseline), Decision Tree (rule-based clarity), Random Forest (primary high-accuracy ensemble), and Gradient Boosting (sequential error reduction). Combining them via weighted probability averaging balances precision and recall.

Q: Why is XGBoost not used in active prediction?

A: Auditing revealed `xgboost.joblib` was a mislabeled GradientBoostingClassifier artifact from an earlier pipeline run. Excluding it ensures strict mathematical accuracy across verified Phase-2 models.

Q: How is the Risk Score calculated?

A: Individual model probabilities are multiplied by ensemble weights (RF 35%, GB 30%, LR 20%, DT 15%), summed into an Ensemble Probability, rounded to 0-100, and mapped to 5 risk levels (LOW, GUARDED, MEDIUM, HIGH, CRITICAL).

Q: What dataset was evaluated?

A: A synthetic development dataset containing 1,200 samples (600 legitimate, 600 phishing). Real-world zero-day validation was not performed in the current study to maintain academic honesty.

Q: What database and ORM are used?

A: SQLite database (`database/phishguard.db`) with SQLAlchemy ORM managing 4 tables: `scans`, `cases`, `audit_logs`, and `research_experiments`.